React Native Important- MSE 1

1. Explain the different types of Mobile application development platforms. Compare Native hybrid and Web Applications with suitable examples.

Mobile application development can be broadly categorized into **Native, Hybrid, and Web application development**. Each approach differs in terms of performance, development cost, platform support, and access to device features.



App Nativo | Web Application | Hybrid App

1. Native Applications

Native applications are developed specifically for a particular operating system using its platform-specific languages and tools. They provide high performance and deep access to device hardware.

Technologies:

- Android → Java, Kotlin, Android Studio
- iOS → Swift, Objective-C, Xcode

Advantages:

- High performance and speed
- Full access to hardware such as camera, GPS, Bluetooth and sensors
- Smooth user experience

- Good security and offline support

Disadvantages:

- Separate codebases are required for Android and iOS
- Higher development cost and time
- More maintenance
- Requires platform-specific knowledge

Examples: Google Maps, Telegram, Pokémon GO.

---

2. Hybrid Applications

Hybrid applications combine web technologies with a **native application container**. A single codebase can be used to deploy the application on multiple platforms.

Common technologies include **HTML, CSS and JavaScript**, together with hybrid frameworks such as Ionic or Cordova.

Advantages

- Single codebase for multiple platforms
- Lower development cost
- Faster development
- Easier maintenance

Disadvantages

- Performance may be lower than fully native applications
- Hardware access may depend on plugins or framework support
- Complex applications may not provide the same experience as native apps

Example: An application developed using Ionic and deployed on both Android and iOS.

---

3. Web Applications

Web applications are applications accessed through a **web browser** rather than being installed as traditional mobile applications. They generally use web technologies such as HTML, CSS and JavaScript.

Advantages

- No installation is required
- Works across different platforms and devices

- Lower development and maintenance cost
- Easy to update
- Broad reach through browsers

Disadvantages

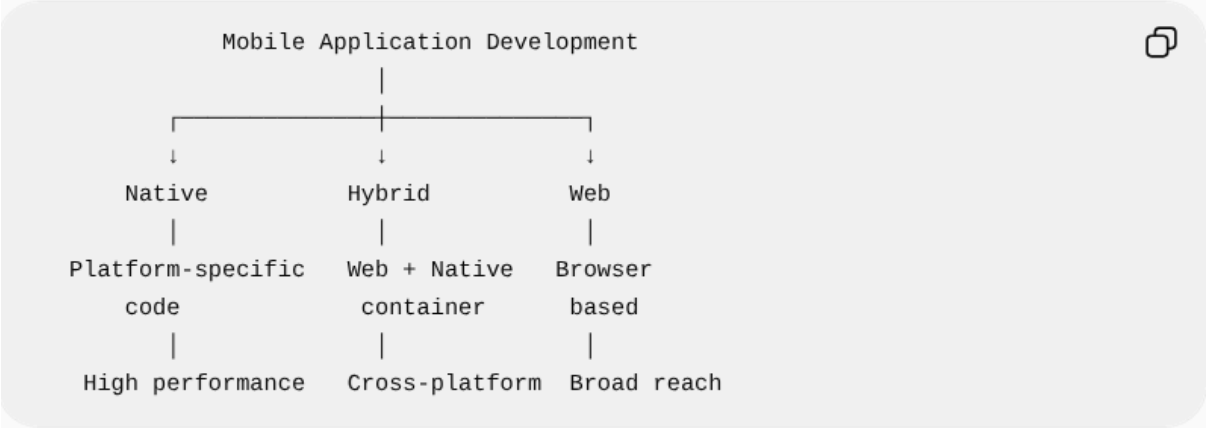
- Requires a browser and generally an internet connection
- Limited access to some device hardware features
- Performance may be lower than native applications

Example: A mobile-friendly banking website accessed through Chrome or Safari.

Comparison of Native, Hybrid and Web Applications

Feature	Native	Hybrid	Web
Development	Platform-specific	Single codebase	Web-based
Performance	Highest	Moderate	Generally lower
Installation	Required	Required	Not required
Hardware Access	Full	Moderate/depends on plugins	Limited
Code Reuse	Low	High	Very high
Development Cost	High	Moderate	Low
Platform Support	Usually one platform	Multiple platforms	Multiple platforms
Example	Google Maps	Ionic app	Mobile banking website

Simple Diagram



Conclusion

Native applications are best when performance and complete hardware access are important. **Hybrid applications** are suitable when cross-platform development and cost reduction are priorities. **Web applications** provide the widest reach with minimal development and installation requirements. The choice depends on the application's **performance, cost, platform, and hardware requirements**.

2. What is JSX , explain the syntax and important features of JSX with suitable examples.

What is JSX?

JSX (JavaScript XML) is a **syntax extension for JavaScript used in React**. It allows developers to write **HTML-like markup inside JavaScript code**. JSX makes React code easier to read, write, and maintain.

JSX is **not HTML and not a programming language**. It is a representation of UI that is converted into JavaScript before execution. Browsers cannot directly understand JSX, so it is transformed into JavaScript, commonly using **Babel**.

Example

Without JSX:

JavaScript

```javascript
const element = React.createElement(
  "h1",
  null,
  "Hello World"
);
```

With JSX:

const element = <h1>Hello World</h1>;

Both represent the same React element, but JSX is much easier to read.

JSX Syntax

The basic syntax is:

const element = <tag attribute="value">Content</tag>;

Example

JavaScript

```javascript
const element = (
  <div title="Outer div">
    <h1>Hello World!</h1>
  </div>
);
```

JSX looks similar to HTML, but it follows some JavaScript/React rules.

Important Features of JSX

1. HTML-like Syntax

JSX allows us to write HTML-like elements directly inside JavaScript.

JavaScript

```javascript
function App() {
  return <h1>Hello Students</h1>;
}
```

This makes UI code simple and readable.

2. JSX Can Be Mixed with JavaScript

JavaScript expressions can be written inside JSX using **curly braces {}**.

```javascript
const name = "Akash";

function App() {
  return <h1>Hello {name}</h1>;
}
```

Output:

Hello Akash

This allows dynamic content to be displayed.

3. JavaScript Expressions

We can use expressions such as calculations, function calls, and variables inside {}.

```javascript
const a = 10;
const b = 20;

function App() {
  return <h1>Sum = {a + b}</h1>;
}
```

Output:

Sum = 30

4. Attributes and Properties

JSX allows attributes to be specified similar to HTML.

```javascript
const element = (
  <img src="photo.jpg" alt="Student" />
);
```

Some HTML attributes use JSX property names. For example, React commonly uses **className instead of class**.

JavaScript

```
<div className="container">
  Hello
</div>
```

5. Nested Elements

JSX supports nesting of elements to create a hierarchy.

JavaScript

```
function App() {
  return (
    <div>
      <h1>Welcome</h1>
      <p>This is React.</p>
    </div>
  );
}
```

This makes it easy to construct complex UI structures.

6. JSX Supports Components

JSX can be used to create and combine React components.

JavaScript

```
function Header() {
  return <h1>My Website</h1>;
}

function App() {
  return (
    <div>
      <Header />
      <p>Welcome!</p>
    </div>
  );
}
```

React applications are built by creating components and combining them to form complete views.

7. Supports JavaScript Logic

Since JSX works together with JavaScript, JavaScript features such as **conditions, loops, and map()** can be used to create dynamic UI.

Example:

```javascript
const students = ["Akash", "Rahul", "Priya"];

function App() {
  return (
    <ul>
      {students.map(student => (
        <li>{student}</li>
      ))}
    </ul>
  );
}
```

8. JSX Is Transformed into JavaScript

The browser does not directly understand JSX. A compiler such as **Babel** transforms JSX into JavaScript/React.createElement() calls before it is executed.

```
JSX
 ↓
Babel
 ↓
JavaScript
 ↓
React
 ↓
UI
```

Advantages of JSX

- **Easy to read and write**
- Makes UI structure clear

- Allows JavaScript and UI markup to be written together
- Supports dynamic content
- Makes component development easier
- Improves code maintainability and productivity

Conclusion

JSX is a JavaScript syntax extension that provides an HTML-like way of describing React UI. It supports JavaScript expressions, attributes, nesting, components, and dynamic content. JSX is ultimately transformed into JavaScript before execution, making it a convenient and readable way to build React user interfaces.

3. Design and explain a simple React Native Application using View , Text , Image and StatusBar. Explain the role of each component.

React Native provides several built-in components for creating mobile user interfaces. A simple application can be designed using **View, Text, Image, and StatusBar** components.

Example Application

Suppose we want to create a simple **Profile Application** displaying a profile image, name, and status bar.

Code

```
import React from 'react';
import {
  View,
  Text,
  Image,
  StatusBar
} from 'react-native';

const App = () => {
  return (
    <View style={{
      flex: 1,
      alignItems: 'center',
      justifyContent: 'center'
    }}>

      <StatusBar barStyle="dark-content" />

      <Image
        source={require('./assets/profile.jpg')}
```

```
      style={{
        width: 150,
        height: 150,
        borderRadius: 75
      }}
    />

    <Text style={{
      fontSize: 24,
      fontWeight: 'bold',
      marginTop: 15
    }}>
      Akash
    </Text>

    <Text style={{
      fontSize: 18,
      marginTop: 10
    }}>
      React Native Developer
    </Text>

  </View>
 );
};

export default App;
```

The material demonstrates the use of View as a container and Text as its child components. The Image component displays local images using require() and normally requires explicit width and height.

Role of Each Component

1. View

View is the **basic container component** in React Native. It is used to group and arrange other components.

JavaScript

```
<View>
  <Text>Hello</Text>
</View>
```

In the above application, View acts as the **parent container** for the StatusBar, Image, and Text components.

It also supports layout and styling properties such as flex, alignItems, and justifyContent.

2. Text

Text is used to **display text on the mobile screen**.

<Text>Akash</Text>

It can be styled using properties such as:

- fontSize
- fontWeight
- textAlign
- fontStyle
- lineHeight

In our application, Text displays:

Akash
React Native Developer

3. Image

Image is used to **display images** in a React Native application.

JavaScript

```
<Image
  source={require('./assets/profile.jpg')}
  style={{
    width: 150,
    height: 150
  }}
/>
```

Images can be loaded from:

- **Local files** using require()
- **Remote URLs** using source={{ uri: '...' }}

In the example, borderRadius: 75 makes the 150 × 150 image appear circular.

4. StatusBar

StatusBar controls the **appearance of the system status bar** at the top of the mobile device. The status bar displays information such as:

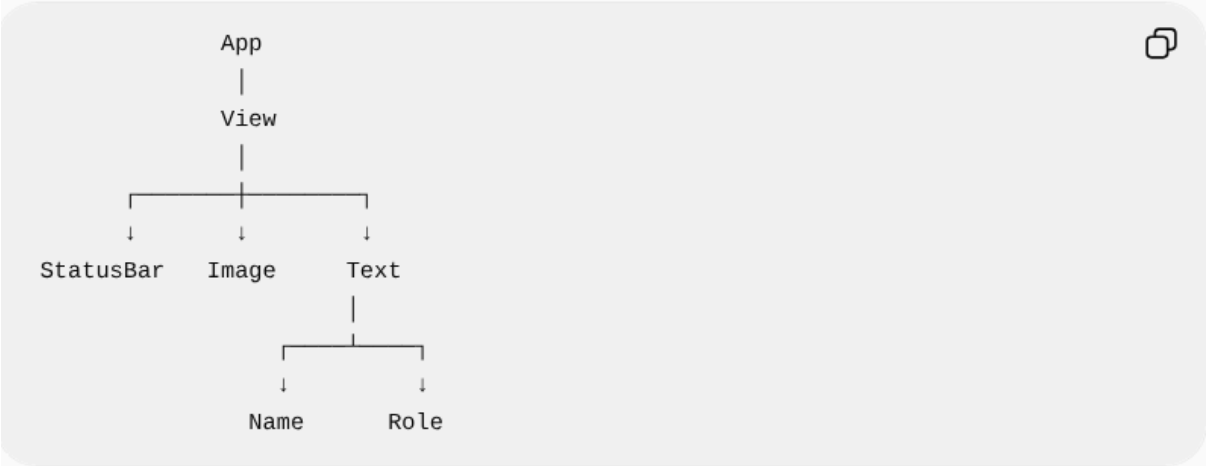
- Time
- Battery level
- Network signal
- Wi-Fi indicator

Example:

<StatusBar barStyle="dark-content" />

dark-content makes the status bar icons and text dark, which is suitable for a light background.

Application Structure



Summary

Component	Role
View	Container used to arrange other components
Text	Displays text on the screen
Image	Displays local or remote images
StatusBar	Controls the appearance of the device's status bar

Conclusion

A React Native application is constructed using a hierarchy of components. **View** provides the layout container, **Text** displays information, **Image** displays visual content, and **StatusBar** controls the system status bar. Together, these components can be used to create a simple and functional mobile application.

4. Explain Logging in React Native, Discuss how console.log() , console.warn() , and console.error() can be used for debugging application.

Logging is the process of displaying information, warnings, and errors while developing an application. It is mainly used for **debugging**, understanding how the application is behaving, and finding problems in the code.

In React Native, the JavaScript console methods **console.log()**, **console.warn()**, and **console.error()** are commonly used for this purpose.

1. console.log()

console.log() is used to print **general information or variable values** to the console.

Example:

```JavaScript
const name = "Akash";
const age = 22;

console.log("Name:", name);
console.log("Age:", age);
```

Output:

Name: Akash
Age: 22

Uses:

- Checking variable values
- Checking whether a function is executing
- Tracking the flow of an application
- Debugging normal application behaviour

2. console.warn()

console.warn() is used to display a **warning** when something may cause a problem or needs the developer's attention.

Example:

```javascript
const age = 15;

if (age < 18) {
  console.warn("User is below 18");
}
```

It helps developers identify **potential problems** without treating them as fatal errors.

Uses:

- Showing potential issues
- Detecting unexpected conditions
- Alerting developers about deprecated or incorrect usage

3. console.error()

console.error() is used to display **errors or serious problems** that need to be investigated.

Example:

```javascript
const login = () => {
  console.error("Login failed");
};
```

It helps developers identify situations where an operation has failed.

Uses:

- Debugging errors
- Identifying failed operations
- Finding problems in API requests or application logic

Example Using All Three

```
import React from 'react';
import { View, Text } from 'react-native';

const App = () => {

 const name = "Akash";

 console.log("App started");
 console.log("User:", name);

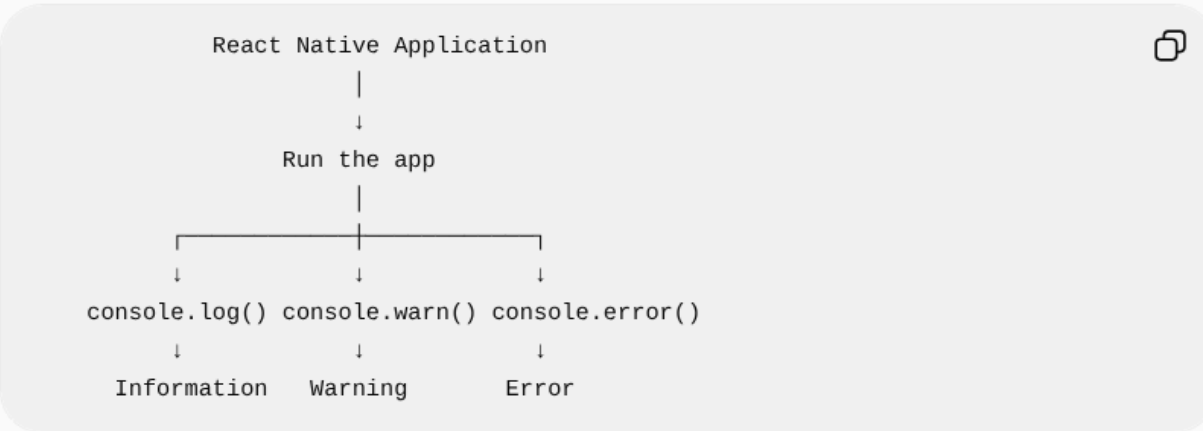
 console.warn("This is a warning");

 console.error("This is an error");

 return (
  <View>
   <Text>Hello {name}</Text>
  </View>
 );
};

export default App;
```

How it helps in debugging



The developer can observe these messages in the **development console/debugging tools** and understand what is happening inside the application.

Difference

Method	Purpose	Example
console.log()	General information	console.log(user)

console.warn()	Potential problem/warning	console.warn("Invalid input")
console.error()	Error or failure	console.error("API failed")

Conclusion

Logging is an important **debugging technique in React Native**. console.log() is used for general information, console.warn() identifies potential problems, and console.error() reports errors. Together, they help developers **understand application behaviour and locate problems during development**.

5. Explain the React Native Architecture , Information flow, and threading model . Illustrate your answer with neat diagram.

React Native architecture defines how JavaScript code communicates with the native platform and how UI components are rendered on Android and iOS. React Native has evolved from the traditional **Bridge-based architecture** to a newer architecture using **JSI, Fabric, and Turbo Modules**.

1. Main Parts of React Native Architecture

a) JavaScript Layer

This is where the application's **JavaScript/React code** runs.

It contains:

- React components
- Application logic
- State management
- Event handling
- API calls

For example, when a user presses a button, the JavaScript code determines what should happen.

b) Native Layer

The native layer consists of platform-specific code for **Android and iOS**.

It is responsible for:

- Rendering native UI

- Accessing device features
- Handling touch and system events
- Communicating with the operating system

c) JSI

JSI (JavaScript Interface) is an important part of the new React Native architecture. It allows **direct communication between JavaScript and native code without depending on the old asynchronous Bridge**.

d) Fabric

Fabric is the new React Native **rendering system**. It improves how UI updates are processed and rendered.

e) TurboModules

TurboModules are the newer native-module system. They support **lazy loading**, which helps reduce unnecessary startup work.

2. Information Flow in React Native

The information flow can be understood as follows:

```
User Interaction
      |
      ↓
Native / UI Thread
      |
      ↓
JavaScript Thread
      |
Application Logic
      |
      ↓
     JSI
      |
      ↓
Native Components
      |
      ↓
Fabric Renderer
      |
      ↓
UI is Updated
```

Step-by-step

Step 1 – User interaction:
The user performs an action such as pressing a button.

Step 2 – Event processing:
The UI thread detects the interaction and sends the event to the JavaScript side.

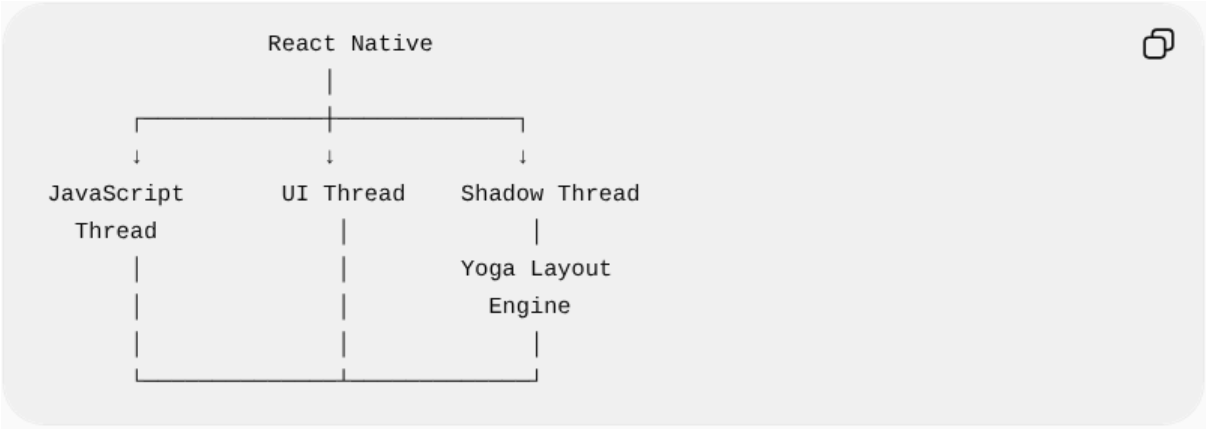
Step 3 – JavaScript processing:
The JavaScript thread executes the application logic, updates state, or performs an API operation.

Step 4 – Communication with native code:
JSI provides the communication mechanism between JavaScript and native code in the new architecture.

Step 5 – Rendering:
The updated UI is processed by the rendering system, **Fabric**, and the native UI is updated.

3. Threading Model

React Native uses **three important threads** to keep applications responsive:



The course material specifically describes these three parallel threads and their responsibilities.

a) JavaScript Thread

The **JavaScript thread** executes the application's JavaScript logic.

It handles:

- State updates
- API calls
- Event handling
- Navigation

- Application logic

Example: When a button is pressed, the JavaScript thread executes the function associated with that button.

b) UI Thread

The **UI Thread** is responsible for **drawing the interface and handling touch events**.

- On Android, it is the **Main Thread**.
- On iOS, it corresponds to the **Main Run Loop**.

It ultimately ensures that the user sees the updated interface.

c) Shadow Thread

The **Shadow Thread** handles layout calculations.

It uses the **Yoga Layout Engine** to calculate layouts based on properties such as:

- flex
- flexDirection
- alignItems
- justifyContent
- Width and height

After calculating the layout, the results are sent to the UI thread.

4. Example of Information Flow

Suppose the user taps a button that changes the size or content of a component.

```
User taps Button
       ↓
   UI Thread
       ↓
 JavaScript Thread
       ↓
State / UI logic changes
       ↓
 Shadow Thread
       ↓
Yoga calculates new layout
       ↓
   UI Thread
       ↓
Updated screen displayed
```

The provided material gives essentially this flow: **UI Thread** → **JavaScript Thread** → **Shadow Thread** → **UI Thread**.

Conclusion

React Native separates application logic, layout calculation, and UI rendering using different threads. The **JavaScript Thread** handles application logic, the **Shadow Thread** calculates layouts, and the **UI Thread** renders the interface and handles user interaction. The newer architecture uses **JSI, Fabric, and TurboModules** to improve communication, rendering, and native-module performance.

6. Explain the jest testing framework and describe how snapshot testing can be used to test React Native components.

What is Jest?

Jest is a JavaScript testing framework used to **automatically test applications and verify whether the code behaves as expected**. It can be used for testing React and React Native components.

Testing helps developers **find bugs early**, before they reach users.

For example, if a function should return 5 when adding 2 + 3, Jest can automatically verify this expected result.

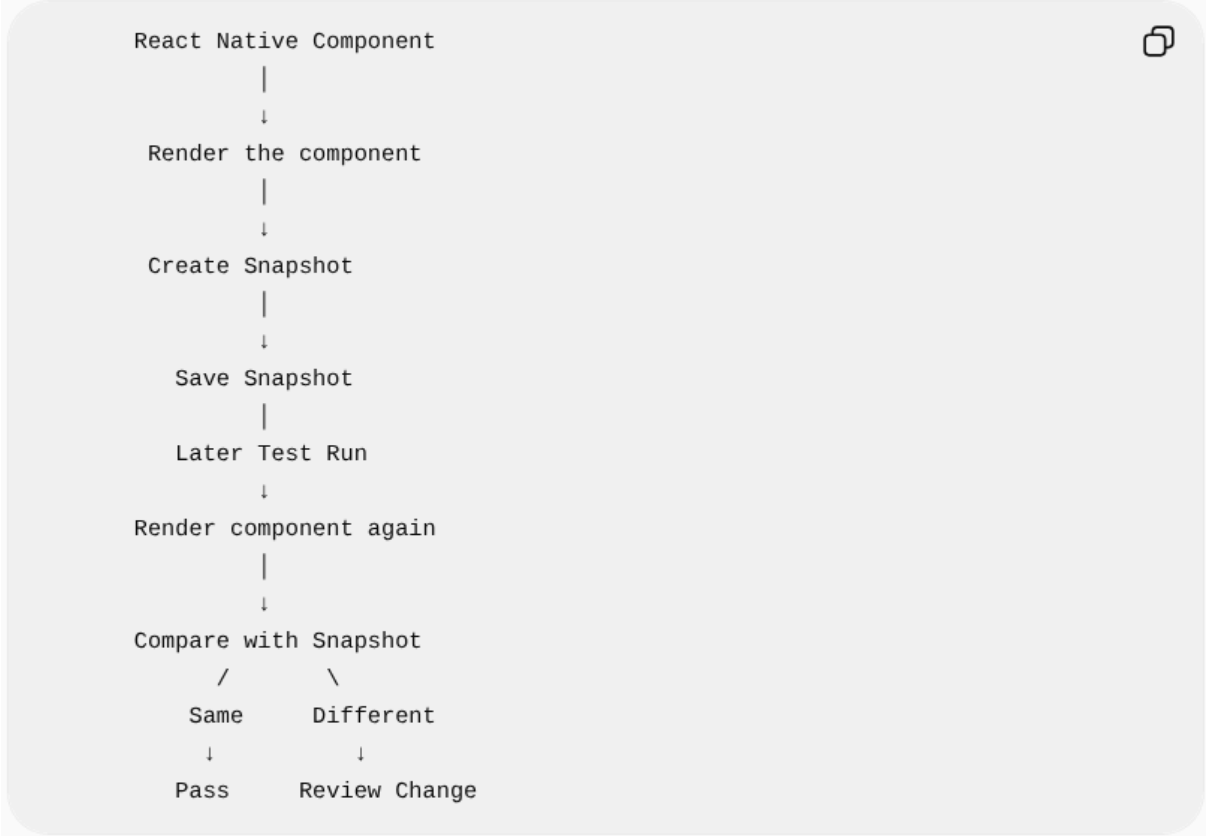
Features of Jest

1. **Easy to use** – Tests can be written using simple JavaScript syntax.
2. **Automatic testing** – Tests can be executed repeatedly whenever the application changes.
3. **Matchers** – Provides functions such as toBe(), toEqual(), etc., to compare expected and actual results.
4. **Component testing** – Can be used to test React and React Native components.
5. **Snapshot testing** – Can save a representation of a component and compare future renders against it.
6. **Bug detection** – Helps identify problems before the application is delivered to users.

Snapshot Testing

Snapshot testing is a testing technique in which Jest takes a **snapshot (stored representation) of a component's rendered output** and saves it.

When the test runs again, Jest compares the **new output with the previously stored snapshot**.



This is useful for detecting **unexpected changes in the UI structure**.

Example

Consider a simple React Native component:

JavaScript

```
import React from 'react';
import { View, Text } from 'react-native';

const Welcome = () => {
  return (
    <View>
      <Text>Welcome to React Native</Text>
    </View>
  );
};

export default Welcome;
```

A Jest snapshot test can be written as:

JavaScript

```
import React from 'react';
import renderer from 'react-test-renderer';
import Welcome from './Welcome';

test('Welcome component matches snapshot', () => {
  const tree = renderer.create(
    <Welcome />
  ).toJSON();

  expect(tree).toMatchSnapshot();
});
```

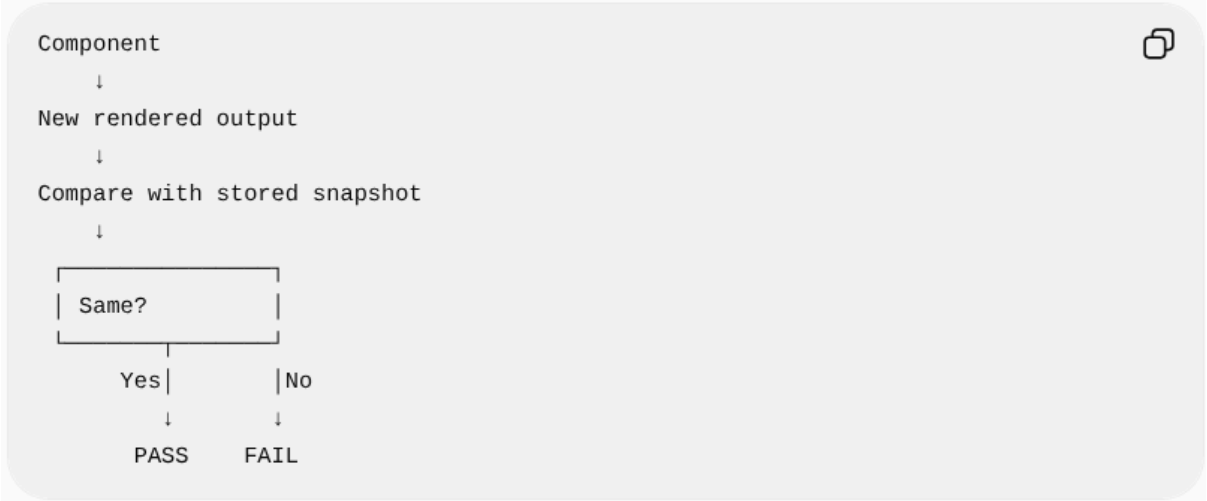
What happens?

First execution:

```
Component
    ↓
Rendered output
    ↓
Snapshot created
    ↓
Snapshot stored
```

Jest creates a snapshot file containing the expected structure of the component.

Later execution:



If someone changes the component unexpectedly, Jest can detect that the new output no longer matches the stored snapshot.

Advantages of Snapshot Testing

- Detects **unexpected UI changes**
- Useful for testing component structure
- Reduces the need to manually check every component
- Helps maintain consistent UI during development
- Makes regression testing easier

Important Point

A snapshot test does **not mean that the UI is visually identical like a screenshot comparison**. It checks the stored representation of the rendered component. Therefore, developers should review snapshot changes and decide whether the change is intentional.

Jest vs Snapshot Testing

Jest	Snapshot Testing
Complete JavaScript testing framework	A testing technique supported by Jest
Tests functions, components, logic, etc.	Mainly checks rendered component output
Uses assertions and matchers	Uses stored snapshots for comparison

Can perform many types of tests	Detects unexpected changes in component structure

Conclusion

Jest is a testing framework that automatically checks whether application code behaves as expected. **Snapshot testing** is one of the techniques that can be used with Jest to test React Native components. It stores the expected rendered output and compares future renders against it, helping developers detect **unexpected changes and UI regressions**.